

Computação Paralela e Distribuída

Ano Lectivo 2025-26

Laboratório - Aula 1

Introdução ao Ambiente Unix

Pré-requisitos para este laboratório:

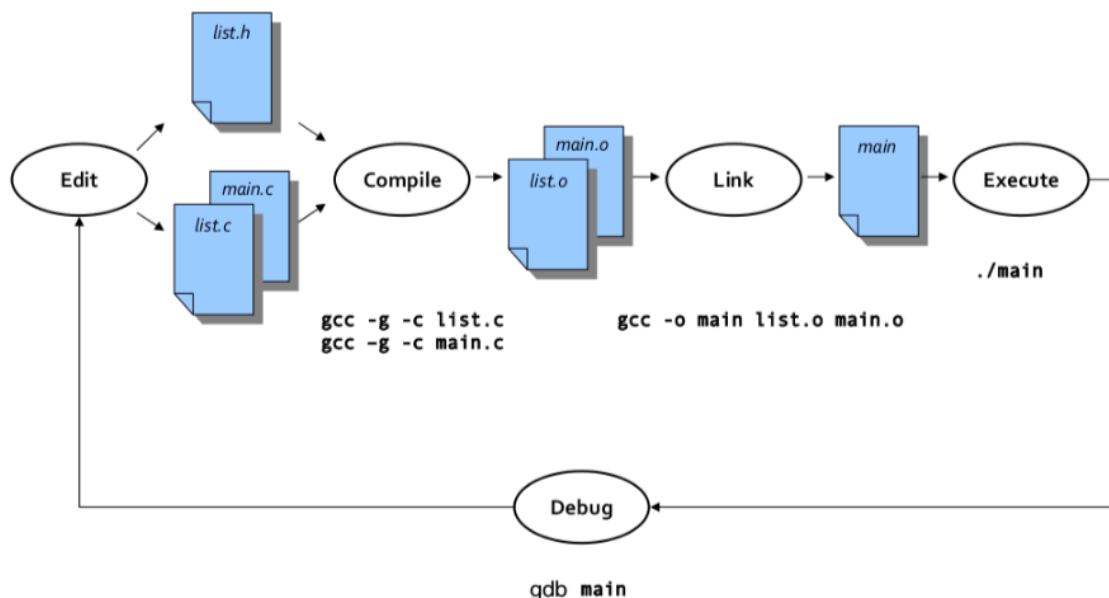
- Já ter instalado o Sistema Operativo Ubuntu e as ferramentas mínimas (gcc, gdb, make) em sua máquina
- Ter revisado as matérias relevantes de Estruturas de Dados e Sistemas Operativos.

1. Objectivos

- Introduzir o desenvolvimento de aplicação em linguagem C no ambiente Unix
- Tornar o estudante familiarizado com as ferramentas necessárias (gcc, gdb, make e comandos Unix).

2. Introdução

Perceba o ciclo de desenvolvimento de aplicações em linguagem C no ambiente Unix:



Copie o ficheiro `aula1-eg1.tgz` para a sua área de trabalho e descomprima o seu conteúdo com o comando:

```
$ tar -zxvf aula1-eg1.tgz
```

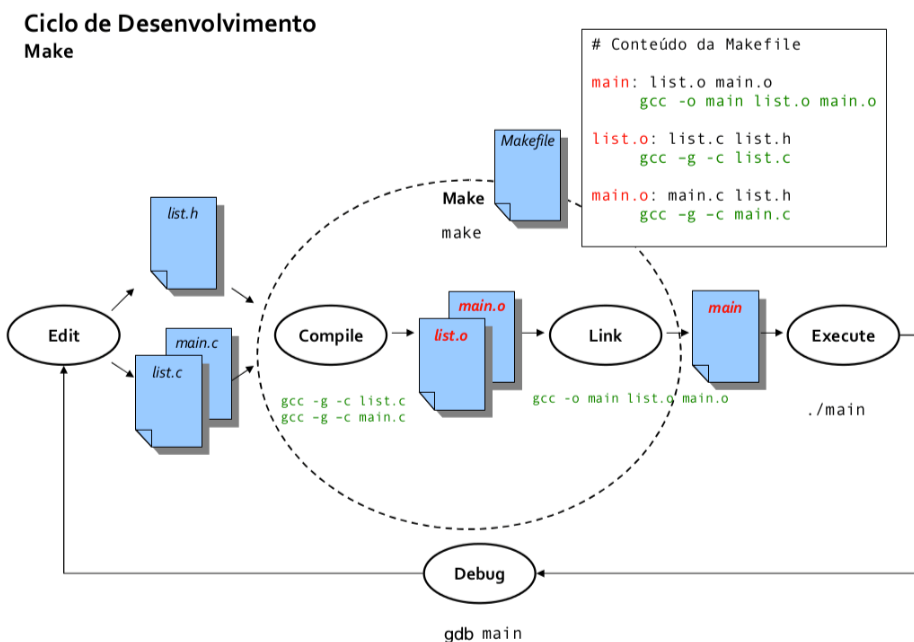
3. Geração do executável

- a) Visualize e compreenda o conteúdo dos ficheiros.
- b) Compile os ficheiros. Verifique depois que os ficheiros objecto foram criados usando o comando "ls" através da interface operacional (*shell*).
\$ **gcc** -g -c list.c main.c
- c) Faça a ligação dos ficheiros objecto de modo a produzir o executável denominado main.
\$ **gcc** -o main list.o main.o
- d) Execute a aplicação main e compreenda o resultado.
\$./main

4. Utilização do debugger gdb

- a) Execute a aplicação no debugger gdb:
\$ **gdb** main
- b) Coloque um breakpoint na primeira instrução da função `insert_new_process` (ficheiro list.c e linha 36):
\$ **b** list.c:36
- c) Execute a aplicação usando o comando run:
\$ **r**
[A aplicação executa-se até ao breakpoint.](#)
- d) Pode agora ver o valor das variáveis que estão no scope da função usando o comando print:
\$ **p** item
[ou](#)
\$ **p** *item
[Qual a diferença entre estes dois comandos?](#)
- e) Pode agora executar a aplicação utilizando o comando de step para executar a próxima instrução entrando dentro das funções:
\$ **s**
[ou o comando de next para executar a próxima instrução sem entrar dentro das funções:](#)
\$ **n**
- f) Enquanto executa os comandos **next** ou **step** pode ir executando o comando print para mostrar o valor das variáveis ou executar apenas uma vez o comando display:
\$ **display** *item
- g) Altere agora o ficheiro list.c: Descomente o comentário da linha 61 e comente a linha 62.
- h) Execute o seguinte comando **fora do gdb** para colocar o limite do tamanho do ficheiro *core* com o valor de 10Mb:
\$ **ulimit** -c 10000000
- i) Gere o executável e execute o programa fora do *gdb*. O erro de *segmentation fault* irá ocorrer.
- j) O gdb é uma muita boa ferramenta para saber o que aconteceu. Para isso, comece por listar os ficheiros que estão na diretoria:
\$ **ls**
[Que observa?](#)

- k) Use em seguida o `gdb` para saber onde ocorreu o erro executando:
- ```
$ gdb main <ficheiro core>
```
- l) O `gdb` irá ficar parado na instrução onde ocorreu o erro. Para saber qual a instrução onde o erro ocorreu execute o comando *backtrace*:
- ```
$ bt
```
- m) O `gdb` mostra-lhe assim a função em que ocorreu o erro e todas as funções que chamaram essa até ao nível da função `main`. Algumas dessas funções são de sistema, mas há duas que podemos reconhecer: *lst_print* e *main*. O primeiro número em cada linha indica o nível em que essa função está. Para observar as variáveis que estão no nível da função *lst_print* deve mudar para esse nível executando o comando *frame* seguido do nível. Por exemplo:
- ```
$ frame 2
```
- n) Pode agora ver o conteúdo das variáveis que estão no scope da função *lst\_print*:
- ```
$ frame 2
```
- Pode assim ver que o erro ocorreu porque a variável `item` é *NULL*. A partir daqui poderia colocar um *breakpoint* no início do ciclo *while*, correr o programa de novo e seguir depois passo a passo, enquanto vai observando os valores das variáveis, até descobrir o que gerou o *segmentation fault*.



4. Utilização da ferramenta make

- Crie o ficheiro *Makefile* na sua área de trabalho e execute *make*. **O que aconteceu?**
- Apague o ficheiro *list.o*. Re-execute *make*. **Interprete o sucedido.**
- Simule uma alteração ao ficheiro *main.c* com o comando seguinte e re-execute *make*. **Compreenda o resultado.**

```
$ touch main.c
```
- Simule a alteração do ficheiro *list.h* e execute *make*. **Porque razão todos os ficheiros foram gerados?**

- e) Simule a alteração do ficheiro *list.o*. O que acontece quando faz *make list.o*? E se agora fizer *make*?
- f) Retire a dependência do ficheiro *list.h* da regra *list.o* da *Makefile*. Repita procedimento da alínea d). Explique a diferença no resultado?
- g) Adicione a regra seguinte no fim do ficheiro. O que descreve esta regra? Identifique: o alvo, as dependências e o comando. Tenha em atenção que os espaços iniciais em cada linha são tabs.


```
clean:
    rm -f *.o main
```
- h) Execute *make clean*. O que aconteceu? Porque razão o comando é executado sempre que esta regra é invocada explicitamente?

4. Outros exercícios

1. Implemente a função `update_terminated_process`. A função `update_terminated_process` recebe uma lista, um valor de `pid` e um tempo de fim, procura pelo elemento com esse valor de `pid` e atualiza esse elemento com o tempo de fim.

2. Gestor de tarefas pessoais

Construa uma aplicação de organização pessoal que permite gerir as tarefas pendentes de uma pessoa. Cada tarefa:

- é identificada por uma sequência de caracteres única que, por simplicidade, não pode conter espaços;
- tem uma prioridade, definida por um inteiro entre 0 e 5 (5 é mais prioritária, 0 é menos prioritária).

A aplicação tem os seguintes comandos:

- `$ new <prioridade> <id-nova-tarefa>`
que insere a nova tarefa;
- `$ list <prioridade>`
que lista todas as tarefas com tarefa da prioridade indicada ou superior; a listagem deve estar ordenada por prioridade (mais prioritárias primeiro) e, entre tarefas igualmente prioritárias, por data de criação (mais recentes primeiro);
- `$ complete <id-nova-tarefa>`
que retira a tarefa indicada; caso a tarefa não exista, deve ser apresentada a mensagem de erro "TAREFA INEXISTENTE".

Sugestão: usar tantas listas quanto níveis de prioridade.

Desafio do Laboratório

Pretende-se desenvolver um terminal paralelo simples, denominado **cpd-terminal**, que executa e monitoriza um conjunto de programas em paralelo em uma máquina multi-core.

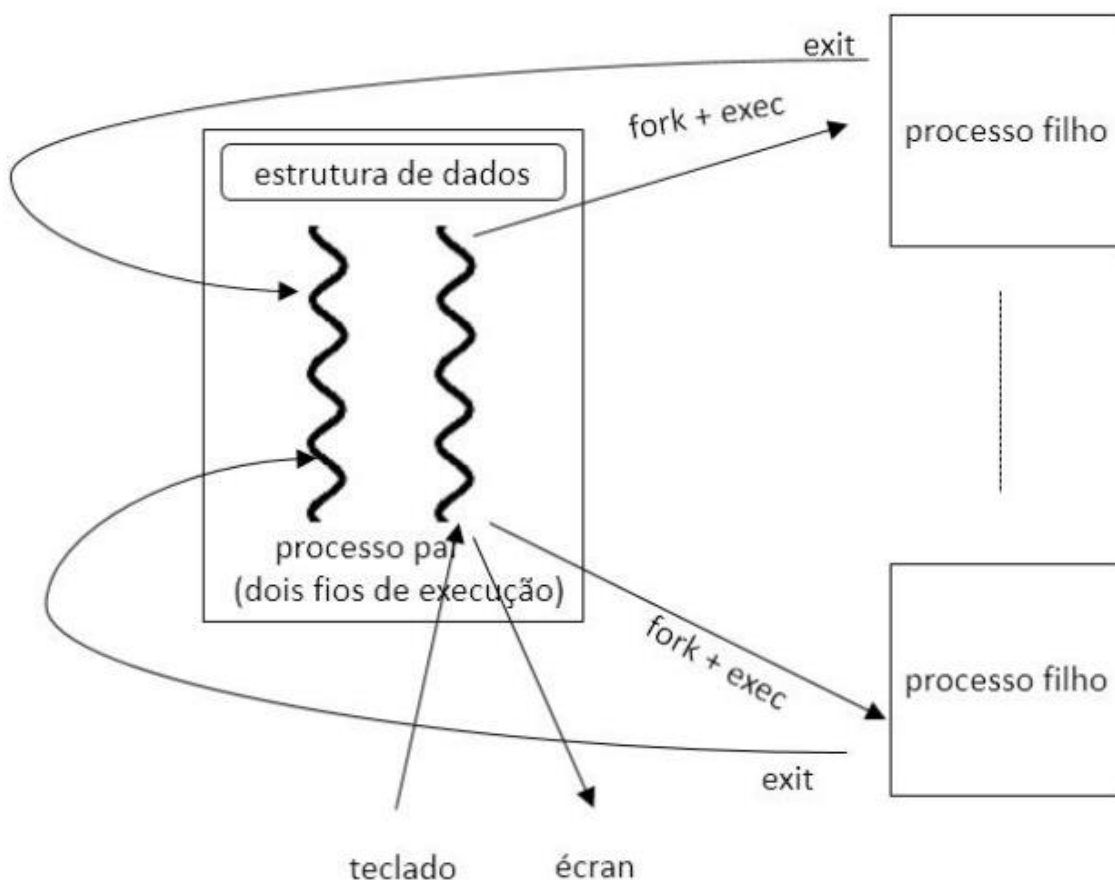


Figura 1: Representação dos processos e de algumas das chamadas sistema a utilizar.

O **cpd-terminal** é um programa que permite a um utilizador emitir ordens para executar programas existentes no sistema de ficheiros da máquina. A característica relevante é de permitir a execução de múltiplos comandos em paralelo.

Etapa 1: Comandos do teclado

Devem ser suportados os seguintes comandos:

- `pathname [arg1 arg2 ...]`, que executa o programa contido no ficheiro indicado pelo `pathname`, como processo filho, passando-lhe os argumentos opcionais que sejam indicados (até um máximo de 5 argumentos permitidos). O processo filho é executado em background, ou seja o **cpd-terminal** não espera pela término dos processos filho que são executados e fica pronta a executar novos processos filho. Caso a execução de algum processo falhe (por exemplo, devido a um `pathname` inválido ou a erro na criação de processo filho), o **cpd-terminal** deve apresentar uma mensagem reportando o erro no `stderr`, mas o **cpd-terminal** não deve terminar.
- `exit`, que termina o **cpd-terminal** de forma ordeira. Em particular, espera pelo término de todos os processos filho (incluindo aqueles que não tenham ainda terminado aquando da ordem de `exit`). Após todos os processos filho terem terminado, o **cpd-terminal** deve apresentar no `stdout` o `pid` e o inteiro devolvido por cada processo filho.

Etapa 2: Monitorização do desempenho

Deve estender o **cpd-terminal** com a funcionalidade de monitorização de desempenho. Para tal, deve ser criada uma tarefa (thread) adicional no **cpd-terminal**, que será responsável por monitorizar os tempos de execução de cada processo filho.

A tarefa de monitorização, denominada tarefa **monitora**, coordena-se com a tarefa principal (que existe, por omissão, sempre em qualquer processo) da seguinte forma:

- A tarefa principal deverá passar a registar o `pid` de cada processo filho executado e o respetivo tempo numa estrutura de dados partilhada (entre as duas tarefas em causa, i.e. a **principal** e a **monitora**). A medição do tempo deverá ser feita usando a função **time**.
- A informação sobre processos filho, partilhada entre ambas as tarefas, deverá ser mantida numa lista simplesmente ligada.
- A tarefa **monitora** espera pela terminação dos processos filho; para cada processo filho terminado, a tarefa **monitora** mede o tempo de terminação e adiciona-o no registo correspondente na estrutura partilhada.

Obviamente, é necessário que ambas as tarefas acima mencionadas se coordenem. Caso contrário, a tarefa **monitora** não sabe se há processos filhos que justifiquem que esta se bloqueie esperando pela sua terminação. Essa coordenação deve ser feita da seguinte forma:

- Além da lista com informação sobre os processos filhos, a tarefa **principal** partilhará com a tarefa **monitora** um inteiro (denominado **numChildren**) que indica o número de processos filho em execução.
- Assim, obviamente, sempre que a tarefa principal lança um novo processo filho (chamando **fork**), incrementa o inteiro **numChildren**.
- A tarefa **monitora** consiste, basicamente, num ciclo infinito; em cada iteração do ciclo, consulta o valor de **numChildren** para saber se há pelo menos um processo filho em execução. Se não há, espera 1 segundo antes da próxima iteração (invocando a função **sleep**). Se há pelo menos um processo filho, espera pela terminação do(s) processo(s) filho, invocando **wait**.

Há um caso que requer especial cuidado: quando o utilizador do **cpd-terminal** dá ordem de terminação (através do comando **exit**), a tarefa **principal** deve informar a tarefa **monitora** para que esta, depois de esperar que todos os processos filho terminem (registando o seu tempo de terminação), também termine. Assim, a tarefa **principal** deverá aguardar que a tarefa **monitora** termine.

Após este ponto de sincronização, antes do processo em causa terminar, a tarefa principal deverá imprimir a informação completa sobre todos os processo filho que foram lançados (e completados) ao longo da execução do **cpd-terminal**: `pid` e tempo de execução em segundos (i.e. tempo de terminação - tempo de lançamento).

Nota importante:

- devem ser usadas as seguintes funções da API do Unix/Linux: `fork`, `exec*`, `wait`, `exit` além das funções habituais da biblioteca `stdio`. E as funções POSIX para gestão de tarefas (threads) e trincos (locks).
- A leitura da linha comandos deve ser feita usando a biblioteca **commandlinereader**, que pode ser obtida na turma Google Classroom.

- A soluçãoo deverá incluir uma **makefile** com um alvo chamado **cpd-terminal** que gera um ficheiro executável da solução, com o mesmo nome. Soluções que não cumpram este requisito não serão avaliadas.

Experiências

Use o programa **fibonacci** fornecido como exemplo para testar o seu **cpd-terminal**. Executando esse programa com argumento elevado, os processos filhos demoram vários segundos a terminar, o que permite testar situações em que múltiplos processos filhos se executam em simultâneo.

Para experiências com múltiplos programas, sugerimos que componha ficheiros de texto com sequências de comandos. Por exemplo:

```
fibonacci 10000
fibonacci 10001
fibonacci 10002
exit
```

Para executar este conjunto de comandos, basta depois executar na linha de comandos: **cpd-terminal** < input.txt (em que input.txt é um ficheiro com uma sequência de comandos tal como a apresentada acima).

Discuta o comportamento da tarefa **monitora**; em particular, considere o caso em que não há processos filho. Altere o valor do tempo de **sleep** e relacione com o conceito de S.O sobre a noção de espera activa.

Bom trabalho!